Arquitectura de Spring Boot explicada con el CU1

Caso de uso 1: Registro de Responsable y creación de Unidad de Venta. Proyecto TP Grupal OO2 - Hito 2 (Epicentro Gourmet).

1. La idea general: arquitectura en capas

Spring Boot organiza la aplicación en capas. Cada capa tiene **una sola responsabilidad** y solo habla con la capa vecina. Eso permite cambiar una parte sin romper las otras.

```
Navegador
   |  (1) HTTP: GET/POST /registro/unidad
   v
[ Controller ]  recibe la petición, valida y elige la vista     RegistroController
   |   usa DTO (datos del formulario)                            RegistroUnidadDTO
   v
[ Service ]     lógica de negocio y transacción                 UnidadVentaService
   |   crea / lee Entities
   v
[ Repository ]  acceso a la base de datos (sin escribir SQL)    IUnidadVentaRepository ...
   |
   v
[ Entity ] <--> Tabla MySQL   (Hibernate/JPA traduce objeto <-> fila)

Respuesta: el Controller devuelve el nombre de una Vista (Thymeleaf) -> HTML al navegador
```

2. Cada pieza: qué es y dónde está en el CU1

Pieza	Qué hace	En el CU1
View (Thymeleaf)	Plantilla HTML. Muestra el formulario y los errores. Se enlaza al DTO con `th:object` y `th:field`.	templates/registro/unidad.html
Controller	Punto de entrada web. Mapea URLs, recibe el DTO, dispara la validación, llama al Service y decide a qué vista ir o redirigir. **No** tiene lógica de negocio pesada ni toca la DB.	RegistroController
DTO (Data Transfer Object)	Objeto simple que transporta los datos del formulario entre la vista y el servidor. Lleva las validaciones de campos. No se guarda en la DB.	RegistroUnidadDTO, TipoUnidad
Service	Lógica de negocio: qué hay que crear y en qué orden. Es *transaccional*: si algo falla, se deshace todo.	IUnidadVentaService / UnidadVentaService
Repository	Interfaz que hereda de `JpaRepository`. Spring genera la implementación: save, findById, existsBy..., sin escribir SQL.	IUnidadVentaRepository, IUserRepository, IPersonalRepository, IFestivalRepository, IUserRoleRepository
Entity	Clase Java mapeada a una tabla (`@Entity`). Cada objeto es una fila. Las relaciones (`@ManyToOne`, etc.) son claves foráneas.	UnidadVenta, FoodTruck, PuestoDesarmable, Responsable, Personal, User, UserRole, Festival
Configuration	Clases que configuran Spring (seguridad, beans).	SecurityConfiguration (permite `/registro/**` sin login, BCrypt)
Helper	Constantes auxiliares para no repetir strings.	ViewRouteHelper.REGISTRO_UNIDAD

3. Recorrido paso a paso del CU1

Paso 1 - El usuario abre el formulario (GET)

El navegador pide `GET /registro/unidad`. El Controller crea un DTO vacío, carga las listas para los combos (festivales y tipos) en el **Model** y devuelve el nombre de la vista.

```
@Controller
@RequestMapping("/registro/unidad")
public class RegistroController {
    @GetMapping
    public String formulario(Model model) {
        model.addAttribute("registro", new RegistroUnidadDTO());   // DTO vacío
        cargarOpciones(model);                                      // festivales y tipos
        return ViewRouteHelper.REGISTRO_UNIDAD;                     // "registro/unidad" -> unidad.html
    }
```

El **Model** es la "mochila" de datos que el Controller le pasa a la vista. Thymeleaf la lee con `${registro}`, `${festivales}`.

Paso 2 - La vista dibuja el formulario

```
<form th:action="@{/registro/unidad}" th:object="${registro}" method="post">
   <input type="text" th:field="*{usuario}" ... />
   <div class="invalid-feedback" th:errors="*{usuario}"></div>   <!-- muestra el error del DTO -->
```

`th:object` vincula el form al DTO; `th:field="*{usuario}"` enlaza cada input con el atributo *usuario* del DTO.

Paso 3 - El DTO define qué datos llegan y cómo se validan

```
@Getter @Setter @NoArgsConstructor          // Lombok genera getters/setters/constructor
public class RegistroUnidadDTO {
    @NotBlank(message = "El usuario es obligatorio")
    private String usuario;
    @Size(min = 6, message = "La contraseña debe tener al menos 6 caracteres")
    private String contrasenia;
    @Pattern(regexp = "^[A-Z]{3}[0-9]{7}$", message = "3 letras mayúsculas y 7 dígitos")
    private String codigoUnico;
    private TipoUnidad tipo;                 // FOOD_TRUCK o PUESTO_DESARMABLE
    private String patente;  private int carpasComponen; ...
}
```

Junta datos del **responsable** y de la **unidad** en un solo objeto, porque el formulario es uno solo. Las anotaciones (`@NotBlank`, `@Pattern`, etc.) hacen la **validación de formato**. Se usa un DTO y no la Entity para que la pantalla no dependa del modelo de la DB.

Paso 4 - El usuario envía el formulario (POST)

```
@PostMapping
public String registrar(@Valid @ModelAttribute("registro") RegistroUnidadDTO registro,
                        BindingResult result, Model model) {
    validarReglasDeNegocio(registro, result);      // ¿usuario repetido? ¿código repetido? ¿mayor de edad?
    if (result.hasErrors()) {
        cargarOpciones(model);
        return ViewRouteHelper.REGISTRO_UNIDAD;    // vuelve al form mostrando los errores
    }
    unidadVentaService.registrarConResponsable(registro);
    return "redirect:/login?registrado";           // todo OK: redirige al login
}
```

Anotación	Significado
`@ModelAttribute`	Spring arma el DTO con los campos del formulario.
`@Valid`	Ejecuta las validaciones del DTO (@NotBlank, @Pattern...).
`BindingResult`	Guarda los errores de validación; la vista los muestra con th:errors.
`redirect:`	Manda al navegador a otra URL (evita reenviar el form al refrescar).

Ojo: el Controller también hace **validaciones de reglas** (usuario/código duplicado, mayor de edad, patente obligatoria si es Food Truck) consultando al Service. Esas no se expresan solo con anotaciones.

Paso 5 - El Service ejecuta la lógica de negocio

Se usa una **interfaz** (`IUnidadVentaService`) y una **implementación** (`UnidadVentaService`). El Controller depende de la interfaz, no de la clase concreta.

```
@Service("unidadVentaService")
public class UnidadVentaService implements IUnidadVentaService {

    // Inyección de dependencias: Spring entrega estas instancias por constructor
    public UnidadVentaService(IUnidadVentaRepository unidadVentaRepository, IPersonalRepository personalReposit
            IFestivalRepository festivalRepository, IUserRepository userRepository,
            IUserRoleRepository userRoleRepository, PasswordEncoder passwordEncoder) { ... }

    @Override
    @Transactional                                  // todo o nada
    public void registrarConResponsable(RegistroUnidadDTO dto) {
        Festival festival = festivalRepository.findById(dto.getFestivalId()).orElseThrow(...);

        User user = userRepository.save(new User(dto.getUsuario(),
                        passwordEncoder.encode(dto.getContrasenia()), true));   // 1) usuario, clave hasheada
        UserRole rol = new UserRole(); rol.setUser(user); rol.setRole("ROLE_RESPONSABLE");
        userRoleRepository.save(rol);                                           // 2) rol

        Responsable r = new Responsable(); ... r.setUser(user);
        personalRepository.save(r);                                             // 3) responsable

        UnidadVenta unidad = crearUnidad(dto);   // FoodTruck o PuestoDesarmable según dto.getTipo()
        unidad.setResponsable(r); unidad.setFestivalAsistio(festival);
        unidadVentaRepository.save(unidad);                                     // 4) unidad

        r.setUnidadVenta(unidad);  personalRepository.save(r);                  // 5) enlaza responsable -> uni
    }
}
```

Concepto	Explicación
Inyección de dependencias (IoC)	No hacemos new de los repositorios. Spring crea los objetos (beans) y los pasa por el constructor.
@Service / @Repository / @Controller	Marcan la clase como bean de Spring y le dan un rol (estereotipos).
@Transactional	Las 5 operaciones son una sola transacción: si falla la 4, se deshacen la 1, 2 y 3.
DTO a Entity	El Service traduce los datos del DTO a objetos Entity (acá a mano con setters).
Polimorfismo	`crearUnidad` devuelve un FoodTruck o PuestoDesarmable; ambos son UnidadVenta.
BCrypt	La contraseña nunca se guarda en texto plano: `passwordEncoder.encode(...)`.

Paso 6 - El Repository guarda en la base

```
@Repository("unidadVentaRepository")
public interface IUnidadVentaRepository extends JpaRepository<UnidadVenta, Long> {
    List<UnidadVenta> findByFestivalAsistioId(long festivalId);
    boolean existsByCodigoUnico(String codigoUnico);      // Spring genera el SQL a partir del nombre
}
```

Es solo una **interfaz**: no hay código de implementación. `JpaRepository<Entidad, TipoDelId>` ya trae *save, findById, findAll, delete...*. Además, los **métodos derivados** (`existsByCodigoUnico`) se convierten en consultas según su nombre. Para consultas más complejas se usa `@Query` (JPQL), como en IUserRepository.

Paso 7 - Las Entities se convierten en tablas

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class UnidadVenta {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private long id;
```

```
    @Column(length = 10, unique = true, nullable = false) private String codigoUnico;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "responsable_id") private Responsable responsable;
    @ManyToOne ... private Festival festivalAsistio;
}
@Entity public class FoodTruck extends UnidadVenta { private String patente; private boolean conexionElectrica,
```

Anotación	Efecto en la base
`@Entity`	La clase es una tabla.
`@Id @GeneratedValue`	Clave primaria autoincremental.
`@Column(unique, length)`	Restricciones de columna.
`@ManyToOne + @JoinColumn`	Clave foránea (muchas unidades -> un festival / un responsable).
`@Inheritance(JOINED)`	Herencia: tabla padre (unidad_venta) + tabla hija (food_truck) unidas por id.
`@Getter @Setter` (Lombok)	Genera getters/setters en compilación.

Con `ddl-auto: update` (application.yml) Hibernate crea/actualiza las tablas solo, a partir de las Entities.

Paso 8 - Seguridad y arranque

SecurityConfiguration declara que `/registro/**` es público (sin login) y el resto exige autenticación. Tras registrarse, el Responsable inicia sesión: **UserService** (implementa `UserDetailsService`) busca el User y sus UserRole para que Spring Security conozca su rol.

La clase principal `Tpgrupaloo2Hito2Grupo01Application` (`@SpringBootApplication`) escanea los paquetes, crea todos los beans y arranca el servidor. `DataInitializer` precarga festivales y admin. Los datos de conexión salen de `.env` vía application.yml.

4. Resumen: el viaje completo de una petición

#	Quién	Qué pasa
1	Navegador	Envía POST /registro/unidad con los campos.
2	Spring MVC	Arma el **DTO** y ejecuta @Valid (errores en BindingResult).
3	Controller	Valida reglas extra consultando al Service; si hay errores, vuelve a la **vista**.
4	Service	Dentro de una transacción crea User, UserRole, Responsable y UnidadVenta (**Entities**).
5	Repository	Cada save() genera un INSERT en MySQL vía Hibernate.
6	Controller	Redirige a /login?registrado.
7	Spring Security	El nuevo usuario inicia sesión y se le asigna ROLE_RESPONSABLE.

5. Por qué esta separación

• **Controller delgado:** solo traduce web a negocio. Si mañana hay una API REST, reutiliza el mismo Service.

• **DTO distinto de Entity:** el formulario mezcla datos de varias tablas; la pantalla no debe depender del esquema de la DB.

• **Service con interfaz:** desacopla y permite testear con dobles.

• **Repository sin SQL:** menos código, menos errores.

• **Cada capa se puede cambiar** sin tocar las otras.

6. Cómo aplicarlo a los próximos casos de uso

Para cada CU nuevo (p. ej. alta de empleados): 1) definir/ajustar las **Entities**; 2) crear el **Repository**; 3) crear el **DTO** con sus validaciones; 4) escribir la interfaz y el **Service** con la lógica y @Transactional; 5) crear el **Controller**; 6) crear la **vista** Thymeleaf; 7) autorizar la ruta en **SecurityConfiguration** según el rol.